SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On class
QUESTIONS: **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
   return strlen(str);
}

int main()
{
   char name[20] = "Compiler"

   int count;

   count = findLength();

   if(count > 5
   {
      printf("Length is greater than 5\n");
   }

   int marks[4] = {85, 90, 75, 80};

   printf("%d\n", marks[4]);

   int choice = 2;

   switch(choice)
   {
      case 1:
         printf("Option 1\n");
         break;

      case 2
         printf("Option 2\n");
         break;

      default:
         printf("Invalid Choice\n");
```

```
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Tasks

(a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

SOLUTION –

a,b) Errors in the Program and Classification of Errors

Line No.	**Error**	**Type**	**Reason**
11	char name[20] = "Compiler"	Syntax Error	Missing semicolon (;) at the end of the statement.
15	count = findLength();	Semantic Error	Function findLength() requires one argument (name), but none is passed.
17	if(count > 5	Syntax Error	Missing closing parenthesis) before {.
25	printf("%d\n",	Runtime	Array index 4 is out of bounds. Valid

	marks[4]);	Error	indices are 0–3.
33	case 2	Syntax Error	Missing colon (:) after case 2.
42	printf("%d\n", *ptr);	Runtime Error	Pointer ptr is uninitialized. Dereferencing it causes undefined behavior.
47	float average = total / students;	Runtime Error	Division by zero because students = 0.
51–54	while(i <= 5)	Logical Error	Variable i is never incremented, resulting in an infinite loop.

c) Compilation phase of each Error

Error Category	**Detected During**	**Explanation**
Lexical Errors	Lexical Analysis	Invalid characters or invalid tokens are detected while scanning the source code.
Syntax Errors	Syntax Analysis (Parser)	Incorrect grammar such as missing semicolons, brackets, or colons is detected by the parser.
Semantic Errors	Semantic Analysis	Checks meaning of statements, function arguments, data types, declarations, etc.
Logical Errors	Not detected by compiler	Program compiles successfully but produces incorrect output or infinite loops.
Runtime Errors	Execution Time	Occur while the program is running, such as division by zero, invalid memory access, or array overflow.

d) Corrected Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
   return strlen(str);
}

int main()
{
   char name[20] = "Compiler";

   int count;

   count = findLength(name);

   if (count > 5)
   {
      printf("Length is greater than 5\n");
   }

   int marks[4] = {85, 90, 75, 80};

   printf("%d\n", marks[3]);
```

```
    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2:
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }

    int value = 50;
    int *ptr = &value;

    printf("%d\n", *ptr);

    int total = 100;
    int students = 5;

    float average = (float)total / students;

    printf("Average = %.2f\n", average);

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Output –

e) Three Compiler Features/Warnings

1. **Syntax Checking**
   - Detects missing semicolons, brackets, colons, and other grammar mistakes during compilation.
2. **Compiler Warning Flags (-Wall, -Wextra in GCC)**
   - Warns about uninitialized variables, unused variables, suspicious code, and possible runtime issues.
3. **Static Analysis Tools**
   - Tools such as **Clang Static Analyzer**, **Cppcheck**, or **GCC Static Analyzer (-fanalyzer)** identify memory errors, null pointers, array bounds violations, and potential logical problems before execution.

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int maximum(int x, int y
{
   if(x > y)
```

```
    return x;
  else
    return y;
}

int main()
{
  int marks[5] = {70, 80, 90, 85, 95};

  int max = maximum(marks[0], marks[1]);

  switch(max)
  {
    case 80:
      printf("Maximum is 80\n")
      break;

    case 90
      printf("Maximum is 90\n");
      break;

    default:
      printf("Other Value\n");
  }

  do
  {
    printf("%d\n", max);
    max--;
  while(max > 75);

  if(max == 75)
    printf("Limit Reached\n"
  }

  return 0;
```

Tasks

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

SOLUTION –

a) Syntax Errors

Line No.	**Syntax Error**	**Reason**
3	int maximum(int x, int y	Missing closing parenthesis) before {.
18	printf("Maximum is 80\n")	Missing semicolon (;) after printf() statement.
21	case 90	Missing colon (:) after case 90.
31	while(max > 75);	Missing closing brace } before while in the do-while loop.
34	printf("Limit Reached\n"	Missing closing parenthesis) and semicolon (;).
35	}	Extra closing brace due to the incomplete if statement.
37	return 0;	Missing closing brace } for the main() function after return 0;.

b) Classification of Syntax Errors

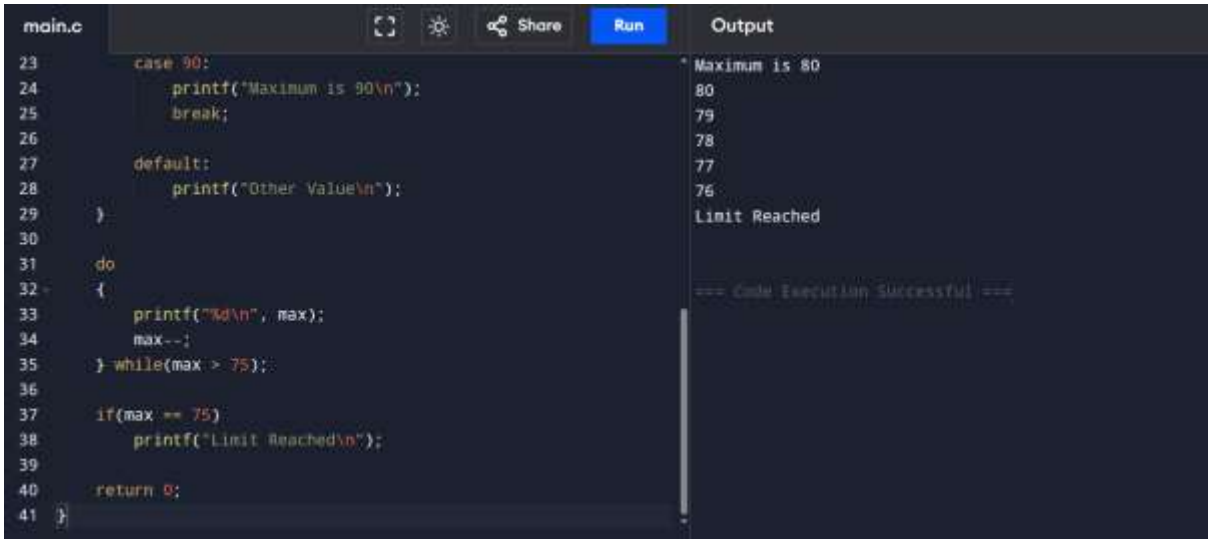
Syntax Error	**Category**
Missing) in function definition	Unmatched Parentheses
Missing ; after printf()	Missing Delimiter
Missing : after case 90	Incorrect Switch-Case Syntax
Missing } before while	Incorrect Loop Structure
Missing) and ; in printf()	Incomplete Statement
Missing closing brace for main()	Unmatched Braces

c) Corrected Program

```
#include <stdio.h>

int maximum(int x, int y)
{
   if(x > y)
      return x;
```

```
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n");

    return 0;
}
```

Output –



d) How the Parser Detects Syntax Errors and Performs Error Recovery?

1. **Syntax Error Detection**
   - The parser (syntax analyzer) receives tokens from the lexical analyzer.
   - It checks whether the tokens follow the grammar rules of the C language.
   - If a grammar rule is violated (such as a missing semicolon, bracket, or colon), the parser reports a syntax error with the corresponding line number.
2. **Error Recovery**

To continue parsing after encountering an error, the parser uses **error recovery techniques**, such as:

- **Panic Mode Recovery:** Skips invalid tokens until a synchronizing token (e.g., ;, }, or)) is found.
- **Phrase-Level Recovery:** Makes small corrections (such as inserting or deleting a token) to continue parsing the remaining program.

These techniques allow the compiler to report **multiple syntax errors in a single compilation** instead of stopping after the first error.

e) Two Improvements for Better Syntax Error Messages

1. **Provide Precise Error Location**

- Report the exact line number and column where the syntax error occurs to help programmers locate the issue quickly.

2. **Suggest Possible Corrections**

- The compiler can provide hints such as:
  - *"Expected ';' after statement."*
  - *"Did you mean ':' after case label?"*
- These suggestions make debugging faster and easier.

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int factorial(int n)
{
   if(n <= 1)
      return 1;

   return n * factorial(n - 1);
}

int main()
{
   int @num = 5;

   int value = 10

   int result;

   result = factorial();

   if(result >= 100)
   {
      printf("Large Number\n");
   }

   switch(result)
   {
      case 24:
         printf("Factorial of 4\n");
         break;

      case 120
         printf("Factorial of 5\n");
```

```
            break;

        default:
            printf("Other Value\n");
    }

    int numbers[5] = {2,4,6,8,10};
    printf("%d\n", numbers[5]);

    int *ptr;
    printf("%d\n", *ptr);

    int a = 50;
    int b = 0;
    int c = a / b;

    int sum = result + 0 + value;

    int i = 1;
    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

SOLUTION –

a,b) Errors in the program and Classification of Errors

Line No.	Error	Error Type	Reason
13	int @num = 5;	**Lexical Error**	@ is an invalid character in a C identifier.
15	int value = 10	**Syntax Error**	Missing semicolon (;).
19	result = factorial();	**Semantic Error**	factorial() requires one argument, but none is provided.
31	case 120	**Syntax Error**	Missing colon (:) after case 120.
39	printf("%d\n", numbers[5]);	**Runtime Error**	Array index 5 is out of bounds. Valid indices are 0–4.
42	printf("%d\n", *ptr);	**Runtime Error**	Pointer ptr is uninitialized. Dereferencing it causes undefined behavior.
46	int c = a / b;	**Runtime Error**	Division by zero (b = 0).
48	int sum = result + 0 + value;	**Code Optimization Issue**	Adding 0 is redundant and can be optimized to result + value.
51–54	while(i <= 5)	**Logical / Runtime Error**	Missing i++, causing an infinite loop.

c) How Compiler Processes the Program?

1. Lexical Analysis

- The lexical analyzer reads the source code and converts it into tokens.
- It detects **invalid identifiers or illegal characters**.
- Example: @num is rejected because @ is not allowed in C identifiers.

2. Syntax Analysis

- The parser checks whether the tokens follow the grammar of the C language.
- It detects errors such as **missing semicolons, colons, parentheses, and braces**.
- Examples:
  - Missing ; after int value = 10
  - Missing : after case 120

3. Semantic Analysis

- The compiler verifies the meaning of statements.
- It checks **function arguments, variable declarations, and type correctness**.
- Example:
  - factorial() is called without the required parameter.

4. Intermediate Code Generation

- After successful semantic analysis, the compiler generates an intermediate representation (IR) of the program.
- Since this program contains earlier compilation errors, IR generation cannot proceed until they are corrected.

5. Code Optimization

- The optimizer removes unnecessary computations and improves efficiency.
- Example:
  - result + 0 + value is simplified to result + value.

6. Runtime

Some errors are detected **only during execution**, such as:

- Array index out of bounds
- Dereferencing an uninitialized pointer
- Division by zero
- Infinite loop

d) Corrected Program

```
#include <stdio.h>

int factorial(int n)
{
   if(n <= 1)
      return 1;

   return n * factorial(n - 1);
}

int main()
{
   int num = 5;

   int value = 10;
```

```
int result;

result = factorial(num);

if(result >= 100)
{
   printf("Large Number\n");
}

switch(result)
{
   case 24:
      printf("Factorial of 4\n");
      break;

   case 120:
      printf("Factorial of 5\n");
      break;

   default:
      printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[4]);

int x = 100;
int *ptr = &x;
printf("%d\n", *ptr);

int a = 50;
int b = 5;
int c = a / b;
printf("c = %d\n", c);

int sum = result + value;
printf("sum = %d\n", sum);

int i = 1;
while(i <= 5)
{
   printf("%d\n", i);
   i++;
}
```

```
    return 0;
}
```

Output –

e) Two Improvements in Compiler Design/ Optimization.

- Advanced **Static Analysis**

  - Static analysis can detect issues such as possible **division by zero**, **array out-of-bounds access**, and **uninitialized pointer usage** before execution.

- Improved **Optimization and Warning Generation**

  - The compiler can eliminate redundant expressions (e.g., +0) and issue warnings for **unused computations**, **infinite loops**, or suspicious constructs by enabling optimization and warning flags (e.g., -Wall, -Wextra, -fanalyzer in GCC).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand